

OE-EC604C — Object Oriented Programming

Semester VI • 3L:0T:0P • 3 Credits

Module 3 Exam Notes: Objects and Classes

CGEC | B.Tech ECE (2023–27) | *Biraj Sarkar*

Contents

1	Classes, Objects and Member Definitions	2
1.1	Specifying a Class and Access Specifiers	2
1.2	Defining Class Member Functions	2
2	Static Data Members and Member Functions	3
2.1	Static Data Members	3
2.2	Static Member Functions	3
3	Friend Functions and Friend Classes	4
3.1	Friend Functions	4
3.2	Friend Classes	6
4	Object Memory and Dynamic Allocation	6
4.1	Memory Considerations for Objects	7
4.2	Dynamic Memory Management: <code>new</code> and <code>delete</code>	7

1. Classes, Objects and Member Definitions

In C++, classes serve as user-defined data types that bind data and operations together, forming the foundation of encapsulation.

1.1 Specifying a Class and Access Specifiers

A class is specified using the `class` keyword. Inside a class, variables (data members) and functions (member functions) are declared under three access levels:

- **private:** Members declared private are accessible only inside member functions of the same class (default access level).
- **public:** Members are accessible from any part of the program where the class object is visible.
- **protected:** Members are accessible within the class itself and its derived (inherited) classes.

1.2 Defining Class Member Functions

Member functions can be defined in two ways:

1. **Inside the Class (Inline Definition):** Functions defined directly within the class declaration are implicitly treated as `inline` by the compiler.
2. **Outside the Class (Scope Resolution):** Functions are declared inside the class and defined outside using the scope resolution operator (`::`).

Member Function Syntax Outside Class

```
Return_Type  Class_Name::Function_Name(Arguments)  {  Body  }  (1)
```

Code Example: Inside vs. Outside class definitions

```
#include <iostream>
class Rectangle {
private:
    double length, width;
public:
    // Defined inside the class: treated as inline
    void setData(double l, double w) {
        length = l;
        width = w;
    }
    double getArea(); // Declared inside class, defined outside
};

// Defined outside using scope resolution
double Rectangle::getArea() {
    return length * width;
}
```

2. Static Data Members and Member Functions

Normally, every object has its own separate copy of all data members. However, static members allow sharing of data across all instances of a class.

2.1 Static Data Members

Static Data Member: A variable declared with the `static` keyword inside a class. Only one copy of the static member is created for the entire class and is shared by all objects of that class, regardless of how many objects are instantiated.

Key Characteristics:

- **Lifetime:** It is initialized to 0 when the first object is created (unless initialized explicitly). It persists throughout the program execution.
- **Storage:** Allocated in the global static memory segment, not within the individual objects.
- **Definition Rule:** It must be defined and initialized outside the class block using the scope resolution operator.

2.2 Static Member Functions

A static member function can only access other static data members or static functions of the same class. **Characteristics:**

- Called using the class name: `ClassName::functionName()`. No object instance is required.

- Does not possess a `this` pointer (since it is not bound to a specific object instance).

Code Example: Tracking Object Counts using Static Members

```
#include <iostream>
class Test {
private:
    int id;
    static int count; // Declaration of static member
public:
    Test() {
        count++;
        id = count;
    }
    void showId() {
        std::cout << "Object ID: " << id << std::endl;
    }
    static int getCount() { // Static member function
        return count; // Can only access static variables
    }
};

// Definition and initialization outside class (mandatory)
int Test::count = 0;

int main() {
    Test t1, t2;
    t1.showId(); // Object ID: 1
    t2.showId(); // Object ID: 2
    std::cout << "Total Objects: " << Test::getCount() << std::endl; // 2
    return 0;
}
```

3. Friend Functions and Friend Classes

To enforce data hiding, non-member functions are strictly blocked from accessing private class attributes. Friendships offer a controlled bypass.

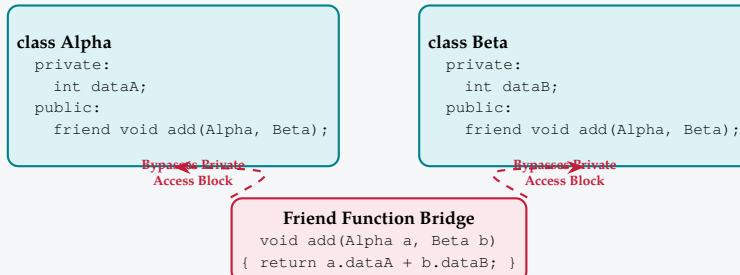
3.1 Friend Functions

Friend Function: A non-member function of a class that is granted permission to access the class's `private` and `protected` members. It is declared inside the class definition with the `friend` keyword.

Special Characteristics of Friend Functions:

- a. It is **not in the scope** of the class to which it has been declared a friend.
- b. It cannot be called using an object of that class (e.g., `obj.friendFunc()` is invalid). It is called like a normal global function: `friendFunc(obj)`.
- c. It usually takes class objects as arguments to process their private fields.
- d. It can be declared in either the `private` or `public` section of the class without altering its behavior.

Friend Function Bridge Bypassing Class Access Restrictions



Code Example: Friend Function accessing private variables of two classes

```
#include <iostream>
class Beta; // Forward declaration required for Alpha's friend prototype

class Alpha {
private:
    int valA;
public:
    Alpha(int v) : valA(v) {}
    friend int add(Alpha, Beta); // Declare friend function
};

class Beta {
private:
    int valB;
public:
    Beta(int v) : valB(v) {}
    friend int add(Alpha, Beta); // Declare same function as friend
};

// Friend function definition (no 'friend' keyword or scope resolution here)
int add(Alpha a, Beta b) {
    return a.valA + b.valB; // Directly accessing private members
}

int main() {
    Alpha a(15);
    Beta b(25);
    std::cout << "Sum = " << add(a, b); // Outputs: Sum = 40
    return 0;
}
```

3.2 Friend Classes

A class can also be declared as a friend of another class. If `class B` is declared as a friend inside `class A`, then **all** member functions of `class B` gain access to the private and protected data members of `class A`.

```
class A { friend class B; }; // Class B is a friend of Class A (2)
```

4. Object Memory and Dynamic Allocation

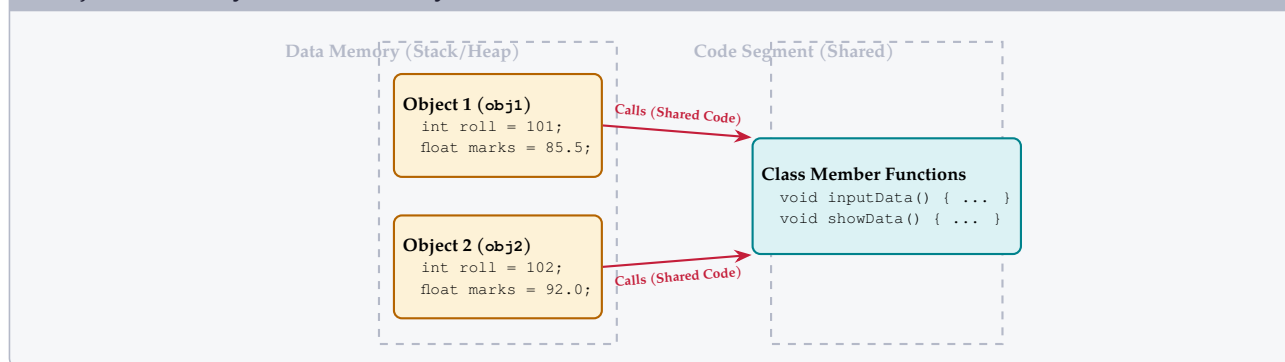
To write efficient code, engineers must understand how objects occupy physical memory and how dynamic memory operators allocate resources on the heap.

4.1 Memory Considerations for Objects

When a class is declared, no memory is allocated. Memory is only allocated when objects of that class are instantiated.

- **Data Members:** Each object receives its own independent memory allocation for non-static data members (to hold distinct object states).
- **Member Functions:** Member functions are allocated only once in the **Code Segment** when the class is loaded. All objects of the class share the exact same memory space for functions. This prevents code redundancy.

Object Memory Allocation Layout in RAM



4.2 Dynamic Memory Management: `new` and `delete`

In addition to C's run-time allocation functions (`malloc/free`), C++ introduces type-safe operators `new` and `delete` which operate directly on the Heap memory.

Dynamic Memory Syntax

Allocate Single Variable:	<code>pointer_var = new data_type;</code>	(3)
Allocate Array of Elements:	<code>pointer_var = new data_type[size];</code>	(4)
Deallocate Single Variable:	<code>delete pointer_var;</code>	(5)
Deallocate Array:	<code>delete [] pointer_var;</code>	(6)

Advantages of `new/delete` over `malloc/free`:

1. **Constructor Calling:** `new` automatically calls the object's constructor; `malloc` only allocates raw bytes.
2. **Destructor Calling:** `delete` automatically calls the destructor before freeing memory; `free` does not.
3. **Type Safety:** `new` returns the exact pointer type (no explicit typecasting needed); `malloc` returns a `void*`.
4. **Safety:** `new` throws a `std::bad_alloc` exception if allocation fails; `malloc` returns `NULL`.

Code Example: Dynamic allocation of variable and array

```
#include <iostream>
int main() {
    // Dynamic single variable allocation
    int *pInt = new int(42); // Allocates int and initializes to 42
    std::cout << "Value: " << *pInt << std::endl;
    delete pInt; // Free memory

    // Dynamic array allocation
    int size = 5;
    int *arr = new int[size]; // Allocates array of 5 integers
    for (int i = 0; i < size; i++) {
        arr[i] = (i + 1) * 10;
        std::cout << arr[i] << " ";
    }
    std::cout << std::endl;

    delete[] arr; // Free array memory (note the square brackets)
    return 0;
}
```


Quick Revision — Module III

Objects and Classes | OE-EC604C

Key Syntax and Configurations

Class Declaration	<code>class MyClass { private: int x; public: void func(); };</code>
External Definition	<code>void MyClass::func() { /* code */ }</code>
Static Definition	<code>int MyClass::staticVar = 0; // Out-of-class definition</code>
Friend Declaration	<code>friend void myFriend(MyClass &obj); // Inside class</code>
Dynamic Single	<code>int *ptr = new int(10); delete ptr;</code>
Dynamic Array	<code>int *arr = new int[5]; delete [] arr;</code>

One-Line Definitions

- **Class:** A user-defined data type that binds data and functions together into a single unit.
- **Object:** An instance of a class that represents a real-world entity and occupies memory.
- **Access Specifiers:** Keywords (`private`, `public`, `protected`) that define the accessibility of class members.
- **Static Data Member:** A class variable shared by all class objects, residing in global static memory.
- **Static Member Function:** A function that can only access static members and is called without an object.
- **Friend Function:** A non-member function declared inside a class that can access its private and protected members.
- **Friend Class:** A class whose member functions can all access the private and protected members of another class.
- **new Operator:** A C++ operator used to dynamically allocate memory on the heap and run constructors.
- **delete Operator:** A C++ operator used to release dynamically allocated heap memory and run destructors.

Frequently Asked / Viva Questions

Q1. What is the default access specifier for class members in C++?

The default access specifier is `private`. If no access specifier is mentioned, members are treated as `private`.

Frequently Asked / Viva Questions (contd.)**Q2. Why do member functions defined inside the class run faster than those defined outside?**

Member functions defined inside the class are implicitly treated as `inline`, meaning the compiler attempts to substitute the call site with the function code, avoiding call overhead.

Q3. Why must static data members be defined outside the class scope?

The class declaration only defines the type, it does not allocate memory. Defining it outside allocates physical storage in global memory.

Q4. Can a static member function access non-static variables of the class?

No. A static function is not bound to any object instance, so it lacks a `this` pointer and cannot determine which object's non-static variable to access.

Q5. Explain why a friend function cannot be called using the object dot (.) operator.

A friend function is not a member of the class. It is a normal global function with special access rights, so calling it with `obj.` causes a compile error.

Q6. Does declaring a friend function inside a class violate encapsulation?

Theoretically yes, as it bypasses private access blocks. However, it is controlled by the class designer and is useful for operations like operator overloading.

Q7. What is the memory size of an empty class object in C++?

It is 1 byte. The compiler allocates 1 byte to ensure that two different object instances of the empty class have unique memory addresses.

Q8. What happens if you use `delete` instead of `delete []` to free a dynamically allocated array?

It results in undefined behavior. Typically, the compiler will only call the destructor for the first element in the array and may fail to release the entire block of memory.

Q9. What is the difference between class and struct in C++?

In a `struct`, members are `public` by default, whereas in a `class`, members are `private` by default.

Q10. Can we declare a constructor static in C++?

No. Constructors are called to initialize specific object instances. A static function has no instance binding, making a static constructor logically impossible.

Q11. Does `new` return a void pointer like `malloc`?

No. The `new` operator is type-safe and returns a typed pointer to the allocated memory, eliminating the need for casting.

Q12. Can we overload a friend function?

Yes, a friend function is a regular function and can be overloaded like any other non-member function.

Comparison: C++ dynamic operators vs. C memory functions

Feature	new / delete (C++)	malloc / free (C)
Type Safety	Type-safe; returns exact type pointer.	Returns <code>void*</code> ; requires explicit typecasting.
Constructors	Calls constructors automatically for classes.	Does not trigger constructors (allocates raw bytes).
Destructors	Calls destructors automatically.	Does not trigger destructors.
Failure Behavior	Throws <code>std::bad_alloc</code> exception.	Returns <code>NULL</code> .
Buffer Size	Automatically calculated by compiler.	Must be explicitly calculated in bytes using <code>sizeof</code> .